

Lab 1D: Ship It

Duration: 35 min

After: Commands, Cost, Permissions & Git slides (Day 1, Block 4)

Prerequisites: Lab 1C complete (modular routes)

Objective

Cycle all four permission modes with Shift+Tab (including **Auto mode**), write explicit permission rules and prove the **deny -> ask -> allow** evaluation order, then ship a feature through a real git branch -> PR workflow.

Deliverable (toolkit chain 4 of 12)

Toolkit repo with a configured `.claude/settings.json` permissions block, a CSV export feature, and a pull request - your first end-to-end ship.

START MARKER: `/clear run; dashboard serving on port 3100.`

Phase 1: Permission Modes - Shift+Tab (8 min)

Press **Shift+Tab** repeatedly and watch the mode indicator cycle:

Default -> AcceptEdits -> Plan -> Auto -> (back to Default)

Mode	Behavior	Use when
Default	Asks before edits and commands	Normal work
AcceptEdits	Auto-approves file edits; still asks for risky commands	Trusted repetitive edits
Plan	Read-only: Claude may plan but not change anything	Architecture decisions, risky areas
Auto	Works autonomously within your permission rules	Well-scoped tasks with guardrails

Step 1 - cycle to **Plan** mode and ask:

Add a simple bar chart to the dashboard showing task counts by status. Plan only.

Expected output marker: a plan, and NO files changed.

Step 2 - cycle to **Auto** mode and let it execute the plan:

Implement the chart plan now.

Expected output marker: Claude builds the chart without permission prompts. Refresh `http://localhost:3100` - chart visible.

Auto mode safety (Jun-Jul 2026): Auto mode ships with two-layer protection - an input injection probe and an output transcript classifier. It blocks transcript tampering and asks before running `rm -rf` on unresolved variables. Autonomy comes fenced, and YOUR permission rules (next phase) are the fence.

Step 3 - Shift+Tab back to **Default** for the rest of the lab.

Phase 2: Permission Rules - deny -> ask -> allow (10 min)

Step 1 - open the menu:

```
/permissions
```

Configure through the interactive menu:

```
Allow (run without asking): Read, Grep, Glob,
Bash(git status:*), Bash(git diff:*), Bash(npm test:*)
Ask (keep default approval): Write, Edit, Bash(git commit:*), Bash(git push:*)
Deny: Bash(rm -rf:*), Bash(sudo:*)
```

Step 2 - learn the evaluation order. In settings.json these become three arrays. Claude Code evaluates **deny first, then ask, then allow** - a deny rule wins even if an allow rule also matches.

Show me the permissions section of .claude/settings.json and explain which rule wins if a command matches both an allow and a deny pattern.

Expected output marker: Claude states deny is evaluated first.

Step 3 - test all three paths:

```
Read server.js and show me the first 10 lines
```

Expected: auto-approved (allow).

```
Add a comment to the top of server.js
```

Expected: asks for approval (ask).

```
Run: rm -rf ./node_modules
```

Expected: **blocked** (deny). Claude explains it cannot run the command.

Phase 3: Price a Session (3 min)

```
/cost
```

Expected output marker: token usage and cost for today. Note the number.

Sonnet 5 pricing: \$2/\$10 per M tokens intro through 2026-08-31, then \$3/\$15. A model dial exists per task - heavier reasoning modes like /fast and /effort settings exist (verify at delivery - fast-moving). Rule of thumb: Sonnet 5 for the loop, Opus 4.8 for gnarly architecture.

Phase 4: Feature Branch -> PR (12 min)

Step 1 - branch:

```
Create a new git branch called "feature/export" and switch to it
```

Step 2 - build:

Implement a CSV export feature with these requirements:

- GET /api/export/metrics?format=csv - downloadable CSV of all metrics
- GET /api/export/tasks?format=csv - downloadable CSV of all tasks
- CSV logic must live in a reusable module (src/utils/csvExport.js)
- Routes remain thin controllers
- Add "Export CSV" buttons to the dashboard UI for both sections
- Follow CLAUDE.md conventions

Step 3 - test:

Test both CSV export endpoints and show me the output

Also click both Export buttons in the browser - each downloads a .csv.

Step 4 - commit and ship a PR:

Commit the export feature with a conventional commit message.

If you have a GitHub remote (from Lab 0 or `gh auth login`):

Push this branch and open a pull request titled "feat: CSV export for metrics and tasks" with a summary of the changes and how to test them.

Expected output marker: Claude runs `git push` and `gh pr create`, returning a PR URL.

No GitHub? Merge locally instead:

Switch back to main and merge the feature branch.

Step 5 - review the history:

Show me the git log with a nice one-line format

Expected output marker: clean conventional commits telling the project's story.

FINISH MARKER - Success Checklist

- [] Cycled all four modes with Shift+Tab and used Plan then Auto deliberately
- [] Permission rules configured: allow/ask/deny all tested
- [] Stated the deny -> ask -> allow evaluation order
- [] `/cost` checked - you can price your day
- [] CSV export built on a feature branch
- [] PR opened (or branch merged locally)
- [] Clean conventional git history

If Stuck

Problem	Recovery
Shift+Tab does nothing	Click into the Claude input box first; in VS Code check the keybinding via <code>/keybindings</code>
Auto mode still asks	Some actions fall outside your allow rules - that's the design; add a rule or approve
Deny rule not blocking	Check exact syntax in <code>/permissions</code> - patterns are case-sensitive

Problem	Recovery
gh not authenticated	Run <code>!gh auth login</code> or fall back to local merge
Chart/export broke the UI	Double-Esc, rewind code to the last good checkpoint, retry with a tighter prompt

Debrief Questions

1. When is Auto mode appropriate - and what makes it safe enough to use?
2. Why must deny be evaluated before allow?
3. What was in your PR description that a reviewer would thank you for?